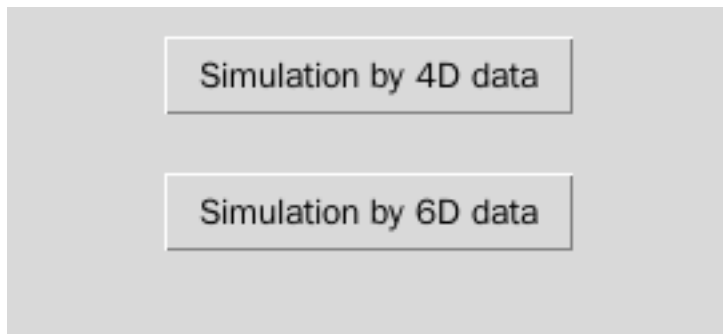


一 加分題完成功能

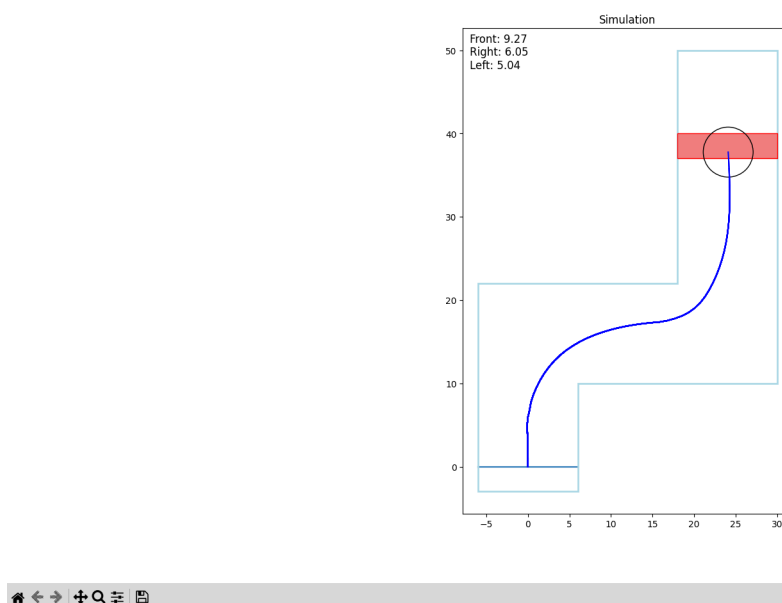
☒ 自行編寫模擬程式

二 程式介面說明

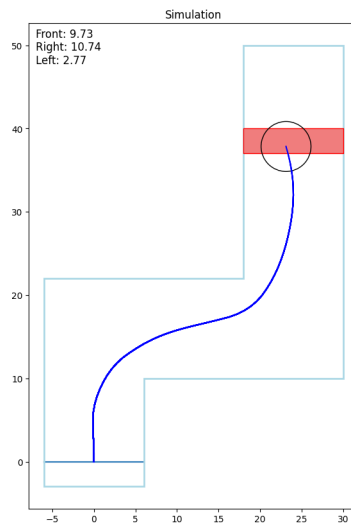


兩個選項可以選擇用 `train4dAll.txt` 或是 `train6dAll.txt` 訓練

三 實驗結果



`train4dAll.txt`



⌂ ⏪ ⏩ 🔍 📄

train6dAll.txt

四 程式說明

四.1 多層感知機

四.1.a Min-Max

```
def _minmax(self, X, min, max):
    self.min = (np.min(X, axis=0), min)
    self.max = (np.max(X, axis=0), max)

def _minmax_transform(self, x, index):
    return (x - self.min[index]) / (self.max[index] - self.min[index])

def _minmax_transform_inverse(self, x):
    return x * (self.max[1] - self.min[1]) + self.min[1]
```

$$x' = \frac{x - \min}{\max - \min}$$

$$x = x' \times (\max - \min) + \min$$

四.1.b 前饋(Forward)

```
def _forward(self, input):
    activations = [input]
    for i in range(len(self.weights) - 1):
        x = np.insert(activations[-1], 0, -1).reshape(-1, 1)
        v = self.weights[i] @ x
        y = self._activation(v)
        activations.append(y)
    x = np.insert(activations[-1], 0, -1).reshape(-1, 1)
    v = self.weights[-1] @ x
    activations.append(v)
    return activations
```

$$y(n) = \varphi[w^T(n)x(n)]$$

四.1.c Backward

```
def _backward(self, activations, y):
    grads_W = [None] * len(self.weights)

    delta = y - activations[-1]
    grads_W[-1] = np.outer(delta.reshape(-1), np.insert(activations[-2], 0, -1))

    for i in reversed(range(len(self.weights) - 1)):
        delta = self._activation_derive(activations[i + 1]) * (self.weights[i + 1][:,
1:].T @ delta)
        grads_W[i] = np.outer(delta.reshape(-1), np.insert(activations[i], 0, -1))

    return grads_W
```

$$\delta_j = e_j(n) \varphi'(v_j(n))$$

$$\delta_{j(n)} = \varphi'(v_{j(n)}) \sum_k \delta_k(n) w_{kj}(n)$$

四.1.d 調整鍵結值

```
self.weights[i] += self.learning_rate * grads_W[i]
```

$$w_{ji} = w_{ji} + \Delta w_{ji} = w_{ji} + \eta \times \delta_j(n) \times y_i(n)$$

四.2 模擬程式

四.2.a 模擬車運動

```
def move(self, wheelAngle=None):
    if wheelAngle is not None:
        self.wheelAngle = wheelAngle

    angle_r = math.radians(self.angle)
    wheelAngle_r = math.radians(self.wheelAngle)
    x = self.position[-1].x + math.cos(angle_r + wheelAngle_r) * math.sin(wheelAngle_r)
    * math.sin(angle_r)
    y = self.position[-1].y + math.sin(angle_r + wheelAngle_r) - math.sin(wheelAngle_r)
    * math.cos(angle_r)
    angle = (self.angle - math.degrees(math.asin(math.sin(wheelAngle_r) /
self.radius))) % 360

    self.position.append(Point2D(x, y))
    self.angle = angle
```

$$x(t+1) = x(t) + \cos[\varphi(t) + \theta(t)] + \sin[\theta(t)] \sin[\varphi(t)]$$

$$y(t+1) = y(t) + \sin[\varphi(t) + \theta(t)] - \sin[\theta(t)] \cos[\varphi(t)]$$

$$\varphi(t+1) = \varphi(t) - \arcsin\left[\frac{2 \sin[\theta(t)]}{b}\right]$$

四.2.b 判斷點是否在多邊形內

```
def inArea(self, point: Point2D) -> bool:
    winding_number = 0
    for edge in self.edges:
        if edge.point1.y <= point.y < edge.point2.y and edge.isLeft(point) > 0:
            winding_number += 1
        elif edge.point2.y < point.y <= edge.point1.y and edge.isLeft(point) < 0:
            winding_number -= 1
    return winding_number != 0
```

四.2.c 判斷點的射線與多邊形邊的距離

```
def ray_intersection(self, point: Point2D, direction: Point2D) -> float:
    min_scaler = float("inf")

    for edge in self.edges:
        edge_direction = edge.point2 - edge.point1
        denominator = direction.cross(edge_direction)

        if abs(denominator) < 1e-9:
            continue

        v = edge.point1 - point
        direction_scaler = v.cross(edge_direction) / denominator
        edge_scaler = v.cross(direction) / denominator

        if 0 <= direction_scaler < min_scaler and 0 <= edge_scaler <= 1:
            min_scaler = direction_scaler

    return min_scaler * direction.length()
```

五 分析

1. 因為訓練資料較少的緣故，單看預測數字與資料集感覺差距很大，但在實際模擬時效果卻很好
2. 一開始對特徵進行標準化，但因為 ReLU 函數對負數值會直接為 0 導致訓練失敗，因此改成使用 Min-Max 與 Leaky-ReLU
3. 因為初始權重隨機與 ReLU 函數特性(神經元死亡)，有機率會訓練失敗，在使用 Leaky-ReLU 後有減緩問題出現頻率